

CHƯƠNG 2 - DOCKERFILE VÀ TỰ TẠO IMAGE

BÀI 5

CHẠY IMAGE my-web:1.0 THÀNH CONTAINER

Bài học thực hành dùng docker run để tạo Container từ Image, đặt tên Container, publish port và truy cập website qua trình duyệt.

1. Mục tiêu bài học

- Hiểu docker run tạo Container mới từ Image.
- Hiểu -d là detached mode và Container chạy nền.
- Biết đặt tên Container bằng --name.
- Hiểu port mapping với -p HOST_PORT:CONTAINER_PORT.
- Chạy được my-web:1.0 thành Container.
- Kiểm tra Container bằng docker ps và docker ps -a.
- Truy cập website qua localhost:8080.
- Hiểu vòng đời cơ bản: run, stop, start, rm.
- Phân biệt rõ Image và Container qua ví dụ thực tế.

2. Trạng thái trước Bài 5

Sau Bài 4, ta đã có Image:

```
my-web:1.0
```

Nhưng chưa có Container được tạo từ Image này.

```
Source Code ☐  
Dockerfile ☐  
Image ☐ my-web:1.0  
Container ☐
```

3. Lệnh chạy Container

```
docker run -d --name my-web-container -p 8080:80 my-web:1.0
```

Lệnh được tách thành:

```
docker run  
-d  
--name my-web-container  
-p 8080:80  
my-web:1.0
```

docker run

Tạo một Container mới từ Image rồi chạy Container đó.

-d

Viết tắt của detached mode. Container chạy ở chế độ nền, terminal vẫn tiếp tục sử dụng bình thường.

--name my-web-container

Đặt tên Container là my-web-container.

-p 8080:80

Publish Host Port 8080 tới Container Port 80.

my-web:1.0

Image được dùng để tạo Container.

Ghi nhớ: docker build tạo Image; docker run tạo Container mới từ Image và chạy nó.

4. Port Mapping

```
-p 8080:80
```

Cú pháp:

```
-p HOST_PORT:CONTAINER_PORT
```

Trong bài này:

```
Host Port: 8080  
Container Port: 80
```

Nginx bên trong Container lắng nghe ở port 80. Trình duyệt trên máy Windows truy cập qua Host Port 8080.

```
Browser  
↓  
localhost:8080  
↓  
Host Port 8080  
↓  
Docker Port Mapping  
↓  
Container Port 80  
↓  
Nginx  
↓  
index.html
```

5. Kết quả thực hành tạo Container

```
PS E:\Docker\Projects\my-web> docker run -d --name my-web-container -p 8080:80 my-web:1.0  
425de6ea88a34392a3b05462d235014476989378f5c48e248045f7e4927d63bf
```

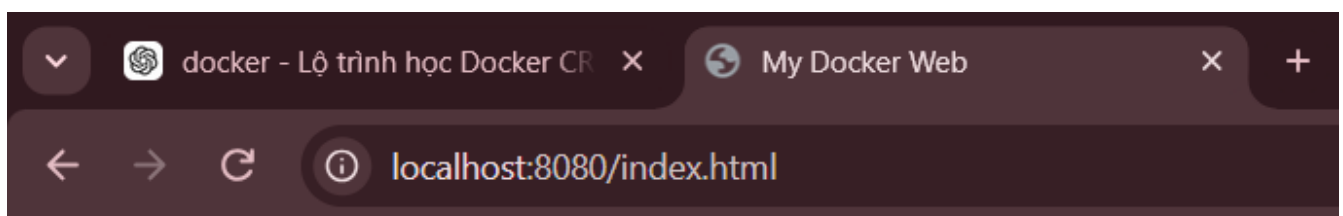
Hình 1. Lệnh docker run chạy thành công và Docker trả về Container ID.

Sau khi lệnh docker run thành công, Docker trả về một chuỗi dài. Đây là Container ID, định danh của Container vừa được tạo.

Container hiện tại có:

```
Image: my-web:1.0  
Name: my-web-container  
Host Port: 8080  
Port trong Container: 80
```

6. Truy cập website trên trình duyệt



Hello Docker!

Đây là website đầu tiên tôi chuẩn bị để Dockerize.

Hình 2. Website trong Container được truy cập thành công qua localhost:8080.

Trang hiển thị đúng nội dung index.html của project. Điều này xác nhận toàn bộ chuỗi Source -> Dockerfile -> Image -> Container -> Port Mapping -> Browser đã hoạt động đúng.

Có thể truy cập:

```
http://localhost:8080
```

Không nhất thiết phải ghi /index.html vì Nginx mặc định phục vụ file index.html.

7. Toàn bộ luồng hoạt động

```
index.html
+
Dockerfile
↓
docker build
↓
my-web:1.0
↓
docker run
↓
my-web-container
↓
-p 8080:80
↓
Nginx
↓
Browser
```

8. Image và Container khác nhau

Image

my-web:1.0 là mẫu đã được build. Nó chứa Nginx và website của bạn nhưng không phải là tiến trình đang chạy.

Container

my-web-container là một instance được tạo từ Image và có thể ở trạng thái Running hoặc Exited.

```
Image: my-web:1.0
↓ docker run
Container: my-web-container
```

Một Image có thể tạo nhiều Container khác nhau.

```
my-web:1.0
├─ my-web-1
├─ my-web-2
└─ my-web-3
```

9. Kiểm tra Container

```
docker ps
```

Hiển thị các Container đang chạy.

Khi Container đang chạy, phần PORTS thường có dạng:

```
0.0.0.0:8080->80/tcp
```

Ý nghĩa: Host Port 8080 được map tới Container Port 80.

```
docker ps -a
```

Hiển thị tất cả Container, gồm cả Running và Exited.

10. Dừng Container

```
docker stop my-web-container
```

docker stop chỉ dừng Container, không xóa Container.

```
Running
↓ docker stop
Exited
```

Sau khi dừng, docker ps không còn hiển thị Container, nhưng docker ps -a vẫn thấy nó.

Lưu ý: Port mapping vẫn là cấu hình của Container, nhưng khi Container đã dừng thì Nginx không còn chạy nên trình duyệt không nhận được phản hồi.

11. Khởi động lại Container

```
docker start my-web-container
```

docker start khởi động lại một Container đã tồn tại. Nó không tạo Container mới.

```
docker run
→ tạo Container mới + chạy

docker start
→ chạy lại Container đã tồn tại
```

12. Xóa Container

Trước tiên nên dừng Container:

```
docker stop my-web-container
```

Sau đó:

```
docker rm my-web-container
```

Container bị xóa nhưng Image my-web:1.0 vẫn còn.

Ghi nhớ: docker rm xóa Container; docker rmi mới dùng để xóa Image.

13. Nếu Image chưa có khi docker run

Khi chạy một Image có thể tải từ Registry nhưng local chưa có, Docker thường sẽ pull Image trước rồi mới tạo Container.

```
Không có Image local
↓
docker run
↓
pull Image từ Registry
↓
tạo Container mới
↓
chạy Container
```

Lưu ý: Điều này chỉ xảy ra nếu Docker có thể tìm và tải Image tương ứng từ Registry. Với Image tự build như my-web:1.0, Image phải tồn tại local hoặc được push lên một Registry trước.

14. Nhiều Container từ cùng một Image

```
docker run -d --name my-web-1 -p 8080:80 my-web:1.0
docker run -d --name my-web-2 -p 8081:80 my-web:1.0
```

Hai Container đều có thể sử dụng Container Port 80 vì chúng có môi trường network riêng. Nhưng Host Port phải khác nhau nếu cùng bind trên host.

```
my-web:1.0
├─ my-web-1 → Host 8080 : Container 80
└─ my-web-2 → Host 8081 : Container 80
```

15. Lỗi thường gặp

Tên Container đã tồn tại

Nếu chạy lại docker run với --name my-web-container trong khi Container cũ vẫn tồn tại, Docker sẽ báo tên Container đang được sử dụng.

Nếu chỉ muốn chạy lại Container cũ:

```
docker start my-web-container
```

Nếu muốn tạo lại từ đầu, dừng và xóa Container cũ trước.

Host Port đã được dùng

Nếu port 8080 đã được Container hoặc chương trình khác sử dụng, Docker không thể bind cùng port đó.

Có thể chọn port khác, ví dụ:

```
-p 8081:80
```

16. Câu hỏi kiểm tra và đáp án

Câu 1. docker run tạo ra Image hay Container?

docker run tạo một Container mới dựa trên Image rồi chạy Container đó.

Câu 2. my-web:1.0 trong lệnh docker run là gì?

Là Docker Image được dùng để tạo Container; my-web là tên Image và 1.0 là Tag.

Câu 3. my-web-container là gì?

Là tên của Container, được đặt bằng tùy chọn --name.

Câu 4. -d dùng để làm gì?

Là detached mode, cho Container chạy ở chế độ nền.

Câu 5. Trong -p 8080:80, 8080 là gì?

Là Host Port.

Câu 6. Trong -p 8080:80, 80 là gì?

Là Container Port.

Câu 7. docker stop có xóa Container không?

Không. Container vẫn tồn tại nhưng chuyển sang trạng thái Exited.

Câu 8. docker start khác docker run thế nào?

docker start chạy lại Container đã tồn tại. docker run tạo một Container mới từ Image rồi chạy nó.

Câu 9. Xóa Container có làm mất Image không?

Không. Container và Image là hai đối tượng khác nhau.

Câu 10. Một Image có thể tạo nhiều Container không?

Có. Một Image có thể được dùng để tạo nhiều Container độc lập.

17. Bài tập thực hành

Bài tập 1 - Kiểm tra Container đang chạy:

```
docker ps
```

Tìm my-web-container và đọc cột IMAGE, PORTS, NAMES.

Bài tập 2 - Dừng Container:

```
docker stop my-web-container
```

Sau đó chạy docker ps và docker ps -a để so sánh.

Bài tập 3 - Kiểm tra trình duyệt khi Container dừng:

Truy cập <http://localhost:8080> và quan sát kết quả.

Bài tập 4 - Khởi động lại Container:

```
docker start my-web-container
```

Sau đó mở lại <http://localhost:8080>.

Bài tập 5 - Tạo Container thứ hai:

```
docker run -d --name my-web-2 -p 8081:80 my-web:1.0
```

Truy cập cả localhost:8080 và localhost:8081, rồi giải thích vì sao hai Container đều dùng được Container Port 80.

18. Tóm tắt câu lệnh và cú pháp

Tạo và chạy Container

```
docker run -d --name my-web-container -p 8080:80 my-web:1.0
```

Tạo Container mới từ my-web:1.0, đặt tên, chạy nền và publish port.

Xem Container đang chạy

```
docker ps
```

Xem tất cả Container

```
docker ps -a
```

Dừng Container

```
docker stop my-web-container
```

Khởi động lại Container

```
docker start my-web-container
```

Xóa Container

```
docker rm my-web-container
```

Cú pháp Port Mapping

```
-p HOST_PORT:CONTAINER_PORT
```

19. Thuật ngữ cuối bài

docker run

Lệnh tạo một Container mới từ Image rồi chạy nó.

Container

Instance có thể chạy được tạo từ Docker Image.

Container ID

Chuỗi định danh duy nhất của một Container.

Container Name

Tên dễ đọc của Container, có thể đặt bằng --name.

Detached Mode

Chế độ chạy nền, bật bằng -d.

Host Port

Port trên máy host chạy Docker; trong bài là 8080.

Container Port

Port bên trong Container nơi ứng dụng lắng nghe; trong bài là 80.

Port Mapping

Ánh xạ Host Port tới Container Port bằng -p.

Publish

Đưa một port trong Container ra host để có thể truy cập từ bên ngoài Container.

Running

Trạng thái Container đang chạy.

Exited

Trạng thái Container đã dừng nhưng vẫn còn tồn tại.

docker stop

Dừng Container nhưng không xóa.

docker start

Khởi động lại Container đã tồn tại.

docker rm

Xóa Container.

docker rmi

Xóa Docker Image.

localhost

Trong bài này là máy Windows host; browser truy cập localhost:8080 để đi tới Container qua port mapping.

20. Sơ đồ tổng kết

```
my-web:1.0
↓ docker run
my-web-container
↓
-d → chạy nòng
--name → đặt tên Container
-p 8080:80 → Host 8080 → Container 80
↓
Nginx
↓
index.html
↓
Browser: localhost:8080
```

Ghi nhớ: Sau Bài 5, bạn đã hoàn tất chuỗi cơ bản Source Code -> Dockerfile -> Build Image -> Run Container -> Publish Port -> Browser.

Bài tiếp theo: Chương 2 - Bài 6: Sửa source, rebuild Image và quan sát Build Cache